

InsightEmb: Learning Action-Intent Embeddings for Agentic Insight Retrieval

Tsz Ting Chung¹ Jiangnan Li² Jie Zhou² Dit-Yan Yeung^{1,†} Mo Yu^{2,†}

¹The Hong Kong University of Science and Technology

²WeChat AI, Tencent

Abstract

Self-improving agents accumulate reusable insights from prior trajectories, making retrieval increasingly important for turning accumulated experience into actionable guidance. At each decision step, retrieving the right insight can help the agent progress toward its goal, a setting we refer to as **agentic insight retrieval**. However, existing retrieval methods primarily model semantic similarity, while overlooking whether a retrieved insight resolves the agent’s current decision bottleneck. We propose **InsightEmb**, a contrastive embedding framework that learns transferable progress-oriented retrieval geometry using *only* mathematical reasoning data. InsightEmb jointly learns to align concrete situations with abstract heuristic rules and to cluster reasoning trajectories with similar progress structures. We evaluate InsightEmb on dynamic agent tasks and a static skill-retrieval benchmark. Without any environment-specific training, InsightEmb improves over all these evaluations, surpassing the performance of existing reasoning embedding models. These results suggest that the geometry of state-insight matching can transfer across domains, enabling effective training from publicly available reasoning data without expensive environment-specific supervision.

1 Introduction

LLM-based agents in interactive environments—for example, web navigation (Yao et al., 2022), embodied tasks (Shridhar et al., 2021), and tool use (Schick et al., 2023)—interleave reasoning and action through frameworks such as ReAct (Yao et al., 2023). They must select actions under large action spaces where the optimal strategy depends on abstract reasoning rather than surface-level pattern matching. Providing agents with *insights*, abstract rules distilled from past experience (e.g., “check likely locations before exploring randomly”), has proven effective (Majumder et al.,

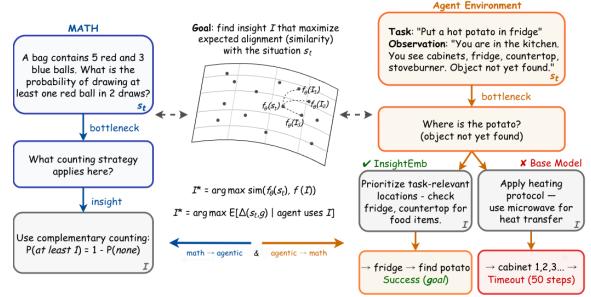


Figure 1: InsightEmb retrieves insights that resolve the agent’s *current* procedural bottleneck rather than topical overlap alone. **Left:** the abstraction gap and shared situation-to-insight structure across math and embodied tasks. **Right:** on an ALFWorld “hot potato” task, Base favors heating rules before the object is found, whereas InsightEmb retrieves search-first guidance.

2023; Wang et al., 2024; Zhao et al., 2024), but the agent must dynamically retrieve the most pertinent insight at each step. We focus on this underspecified retrieval problem: the relevant insight is the one that is *operationally useful now*, given the agent’s current state, goal, and action history.

This dynamic retrieval poses a unique challenge that we call the *abstraction gap*: the query (the agent’s current observation and action history) and the target (an abstract insight) live at fundamentally different levels of abstraction. More importantly, relevance is not determined by topical similarity alone, but by the agent’s *next-step intent*: what bottleneck must be resolved before progress is possible? Standard embedding models match on surface semantics rather than functional relevance such as “the object has not been found, so the useful insight should guide search.” Consequently, off-the-shelf retrievers return topically related but procedurally premature insights, such as state-transformation rules before the agent has located the target object, as illustrated in Figure 1.

Our key observation is that this *situation-to-insight* matching problem is not domain-

specific. In mathematics, a probability query such as “ $P(\text{at least one red})$ in two draws” must be matched to complementary-counting heuristics with no topical overlap with the problem statement. In an embodied task, the same formalism applies: the agent must first resolve the *locate-object* bottleneck before heating or placement insights become useful. Both domains require inferring the latent bottleneck of a concrete situation and retrieving the abstract rule that enables the next effective step.

We propose **InsightEmb**, a contrastive training framework that exploits this domain-agnostic structure. InsightEmb trains an embedding model in two stages, *entirely on publicly available mathematical reasoning data*: (1) *Situation-to-insight matching* teaches the model to bridge the abstraction gap by mapping math problems and (partial) chain-of-thought trajectories to their relevant heuristic rules; (2) *Situation-to-experience matching* teaches structural similarity recognition between reasoning trajectories, reinforcing the model’s ability to identify when different-looking situations require the same underlying strategy. At inference time, the trained model retrieves insights for LLM agents in ALF-World (Shridhar et al., 2021) and WebShop (Yao et al., 2022), and we use SRA-Bench as a static skill-retrieval diagnostic. These evaluations test dynamic state-conditioned retrieval and static skill applicability without any environment-specific training data. Our contributions are threefold:

- We formulate agentic insight retrieval as goal-conditioned action-intent matching. Relevance is defined by whether an insight resolves the current bottleneck and enables progress, rather than by semantic similarity alone.
- We introduce InsightEmb, a cross-domain contrastive training framework for action-oriented retrieval geometry with empirical validation.
- Our empirical results across static and dynamic evaluations show the effectiveness of InsightEmb with fewer steps taken. Analyses reinforce the gains come from action-intended retrieval.

2 Related Work

Retrieval-Augmented LLM Agents. RAG (Lewis et al., 2020; Guu et al., 2020) has been extended to agents that retrieve past experiences (Shinn et al., 2023; Majumder et al., 2023; Packer et al., 2023), tool docs (Qin et al., 2024), and skills (Wang et al., 2024). These systems target *concrete* artifacts where surface similarity

often suffices; we retrieve *abstract* heuristic rules with little lexical overlap, where standard retrievers fail and off-the-shelf or in-domain embedders miss the agentic objective.

Experience-Based Agent Learning. Reflexion (Shinn et al., 2023), CLIN (Majumder et al., 2023), and Voyager (Wang et al., 2024) accumulate episodic memory, causal abstractions, or skills; ExpeL (Zhao et al., 2024) and AutoGuide (Fu et al., 2024) distill trajectories into text rules without weight updates. They focus on *what* to store; we address *when* to retrieve the right stored knowledge and are agnostic to how insights are produced.

Reasoning-Oriented Dense Retrieval. Dense retrievers (Karpukhin et al., 2020; Xiong et al., 2021; Wang et al., 2022; Su et al., 2023; Xiao et al., 2023) and benchmarks such as MTEB (Muenighoff et al., 2023) and BRIGHT (Su et al., 2025) show that strong semantic matching still leaves reasoning-intensive retrieval far from solved. ReasonIR (Shao et al., 2025), Llama-NV-Embed-Reasoning (NVIDIA, 2026) (top open-source *single-model* embedder on BRIGHT), and ReasonEmbed (Chen et al., 2025) train contrastive embedders for static query–passage support. We instead target *next-step utility* under an evolving agent state: whether an insight resolves the current bottleneck, not merely whether it entails a fixed query.

Agentic Benchmarks. ALFWorld (Shridhar et al., 2021), WebShop (Yao et al., 2022), Mind2Web (Deng et al., 2023), and WebArena (Zhou et al., 2024) evaluate embodied and web control with state-conditioned queries; AgentBench (Liu et al., 2024) spans additional interactive environments. SRA-Bench (Su et al., 2026) tests static task-to-skill retrieval over mixed gold and distractor skills, complementing dynamic agent benchmarks by isolating retrieval from online execution.

3 Method

3.1 Problem Setting

An LLM agent operates in an interactive environment where, at each step t , it observes a state s_t (task description, action history, current observation) and must select an action a_t toward a goal g , following a reasoning-and-acting loop (Yao et al., 2023). The agent has access to an insight corpus $\mathcal{I} = \{I_1, \dots, I_N\}$, where each insight I_i is

a natural-language description of abstract rules and strategies. We view this task as *goal-conditioned abductive retrieval*: given the current state and goal, retrieve the insight that best identifies which intervention would make progress possible. Our notion of relevance is *progress-oriented*: an insight is useful if it is expected to reduce the gap to the goal, not merely if it is semantically similar to the query. We express this target as,

$$I^* = \arg \max_{I \in \mathcal{I}} \mathbb{E} [\Delta(s_t, g) \mid \text{agent uses } I], \quad (1)$$

where $\Delta(s_t, g)$ measures progress toward goal g from state s_t . Equation (1) is *not* the training objective; we do not estimate $\mathbb{E}[\Delta \mid I]$ inside the contrastive loss. Instead, it guides *how we curate supervision*: for each training problem we run five rollouts, distill candidate insights from random trajectory subsets, and assign an insight to I^+ for problem q only if prepending it to q ’s query improves solve rate over a no-insight baseline on that same problem (and to I^- otherwise); see Appendix B. At inference, we deploy an embedding retriever that approximates the same criterion by similarity:

$$I^* = \arg \max_{I \in \mathcal{I}} \text{sim}(f_\theta(s_t), f_\theta(I)), \quad (2)$$

with f_θ trained by InfoNCE on the curated (q, I^+, I^-) pairs (§3.3).

The core difficulty is the *abstraction gap*: s_t contains concrete details (specific objects, locations, actions) while I^* contains abstract rules (general strategies, heuristics). For agentic tasks, this gap is also temporal: the relevant insight is the one that addresses the agent’s current bottleneck, not necessarily the one most topically related to the final task goal. For example, before the target object is found, a search insight is more useful than an insight about how to transform or place that object. We address this gap through a two-stage contrastive curriculum trained entirely on mathematical reasoning data (Hendrycks et al., 2021; Wei et al., 2022). Our training design rests on a single claim: math heuristic retrieval and agentic insight retrieval instantiate the same action-intent matching problem as illustrated in Table 1.

3.2 Structural Analogy for Action-Intent Retrieval

We propose that an agentic action-intent embedding can be learned from math-only data because both settings share the same retrieval structure: a

Component	Math	Agentic task
Situation	Problem statement	Task + observation
Trajectory	Solution steps	Action history
Insight	Solving heuristic	Execution strategy
Abstraction gap	“at least one red ball” $\leftrightarrow$ “complementary counting”	“You clean the butterknife” $\leftrightarrow$ “place”

Table 1: Structural analogy between math reasoning and agentic execution—our design claim for building action-intent embeddings from math-only contrastive training.

concrete situation must be matched to the abstract rule that resolves the *current* bottleneck, not merely to topically related text.

Under this parallel, contrastive training on math should teach a domain-agnostic geometric property: embed a situation near the abstract rule that makes the next useful step apparent. The instruction prefix at inference (§3.5) frames queries in domain-neutral terms so this property can activate on ALFWorld and WebShop without environment-specific fine-tuning. The experiments below test whether this claim holds in practice.

3.3 Stage 1: Situation-to-Insight Matching

Stage 1 teaches the embedding model to bridge the abstraction gap by matching mathematical problems to their relevant heuristic rules.

Training data. Each training example is a triplet (q, I^+, I^-) where q is a query in one of three forms: (i) a raw math problem statement (*query-only*), (ii) a problem concatenated with its full chain-of-thought solution (*full trajectory*), or (iii) a problem with a truncated solution (*partial trajectory*). The positive set I^+ contains distilled heuristic rules that pass the utility filter implied by Equation (1): for each problem in a trajectory subset, a rule is in I^+ only if prepending it to that problem’s query improved solve rate on validation attempts for that same problem, compared with attempts without the rule. Rules that fail this check, or hurt performance, are placed in I^- . This ties contrastive labels to *demonstrated progress* rather than author judgment or lexical overlap alone. For a given trajectory subset, the same I^+ and I^- are reused across all three query forms; only the situation anchor q changes. Insights are provided at two granularities used jointly in training: BUNDLE insights (multi-rule summaries distilled from successful and contrasting trajectories) and ATOMIC insights (sin-

gle rules split from those bundles). Stage 1 therefore learns from both coarse strategy sets and fine-grained rules, totaling 11,950 contrastive pairs over counting & probability, number theory, and geometry problems from the MATH dataset (Hendrycks et al., 2021); Appendix E lists pair counts and domain breakdowns.

Training objective. Given curated (q, I^+, I^-) , we train f_θ with InfoNCE (Chen et al., 2020) and in-batch negatives:

$$\mathcal{L}_1 = -\log \frac{\exp(\text{sim}(f_\theta(q), f_\theta(I^+))/\tau)}{\sum_{I \in \{I^+\} \cup \mathcal{N}} \exp(\text{sim}(f_\theta(q), f_\theta(I))/\tau)} \quad (3)$$

where $\tau = 0.01$ and $\mathcal{N}$ includes in-batch negatives (training group size 11). The loss implements Equation (2) on pairs whose positives were chosen by the progress criterion in Equation (1), not by optimizing $\mathbb{E}[\Delta \mid I]$ end-to-end.

3.4 Stage 2: Situation-to-Experience Matching

Stage 1 teaches the model to match situations to *abstract rules*; Stage 2 complements this by teaching it to match situations to *structurally similar solved problems*. This reinforces the model’s ability to recognize when two different-looking situations require the same reasoning approach.

The training data consists of triplets (q, T^+, T^-) where q is a raw problem or a full trajectory, T^+ is a structurally similar solved problem, and T^- is a dissimilar one. The loss uses the same contrastive objective as Stage 1, with trajectories replacing insights. This stage uses 2,896 examples across the same three math domains.

3.5 Inference: Dynamic Insight Retrieval

At inference time, the trained embedding model is deployed for dynamic insight retrieval in target interactive environments *without any domain-specific fine-tuning*. At each step t : (1) the agent’s current state s_t is encoded with a task-specific instruction prefix;¹ (2) the top- k most similar insights are retrieved from the pre-encoded corpus; (3) the retrieved insights are prepended to the LLM’s prompt for action generation.

4 Experimental Setup

We fine-tune Qwen3-Embedding-4B (Zhang et al., 2025) with the two-stage curriculum in §3; Ap-

¹E.g., “Given an *AlfWorld* task observation, retrieve relevant insights or strategies that can help the agent solve the task effectively.”

pendix J specifies learning rate, batching, contrastive temperature, and hardware, and Table 13 summarizes the full configuration.

4.1 Evaluation Overview

Our goal is to test the design claim in Table 1: whether math-only training yields progress-oriented, action-intent retrieval in ALFWorld, WebShop, and SRA-Bench. We use two complementary metrics: *end-task performance* on interactive agent environments and *retrieval recall* on a static benchmark. On ALFWorld and WebShop, an LLM agent executes full trajectories with dynamic insight retrieval; we measure whether retrieval improves task completion (success rate and average task score), not ranking accuracy in isolation. On SRA-Bench, no agent is run. We score task-to-skill matching directly with $\text{recall}@k$ and $\text{nDCG}@k$. ALFWorld stresses embodied search and manipulation; WebShop stresses product comparison and purchase workflows. Following AgentBench (Liu et al., 2024), we report success rate for ALFWorld and average task score for WebShop.

4.2 Evaluation with Agentic Environment

4.2.1 ALFWorld

We evaluate on ALFWorld (Shridhar et al., 2021), a text-based embodied environment where agents perform household tasks. Each task requires a sequence of navigation and interaction actions, making ALFWorld a direct test of whether retrieval can select insights that unblock the agent’s current procedural phase. We use Qwen3-8B (Qwen Team, 2025) as the action-generating LLM with greedy decoding, a history window of 3 steps, maximum 50 steps per game, and top-1 insight retrieval. We evaluate on the test split.

4.2.2 WebShop

To validate dynamic retrieval beyond embodied tasks, we additionally evaluate on WebShop (Yao et al., 2022), where agents navigate product pages to purchase items matching natural-language instructions. This requires retrieving insights for attribute comparison, variant selection, loop recovery, and multi-step purchase workflows. We use the same Qwen3-8B action-generating LLM with greedy decoding, a history window of 3 steps, maximum 50 steps per game, top-1 dynamic insight retrieval, and 4 parallel environments. We evaluate on 500 test games. We compare the same embedding model variants: Base, InsightEmb.

Insight	Embedding	ALFWorld	WebShop
{}	—	52.86	31.03
BUNDLE	Base	54.29	18.42
BUNDLE	InsightEmb	60.71	31.74
ATOMIC	Base	55.00	28.25
ATOMIC	InsightEmb	59.29	32.05

Table 2: Dynamic agent evaluation on ALFWorld and WebShop. Both columns are reported as percentages.

4.2.3 Insight corpora

We evaluate two insight corpus granularities per environment to disentangle the effect of the embedding model from the insight content. BUNDLE insights are multi-rule summaries, each containing several rules with full chain-of-thought reasoning. ATOMIC splits each bundle into individual rules with chain-of-thought removed. Appendix Table 7 summarises corpus sizes and rule lengths for both environments. Appendix B documents trajectory collection, DeepSeek-R1/GPT-5.2 distillation prompts, and how BUNDLE and ATOMIC corpora are built.

4.3 SRA-Bench Retrieval Evaluation

To isolate retrieval quality from downstream action generation, we additionally evaluate on SRA-Bench (Su et al., 2026), spanning theorem proving, logical reasoning, tool use, contest math, medical calculation, and code generation. Each query uses the full instance question field with a Qwen3-style retrieval instruction prefix (Zhang et al., 2025); each candidate encodes the skill name, description, and body. Appendix K gives the exact Instruct/Query template and skill-passage format used in evaluation. Unlike the agent environments above, the primary metrics here are retrieval recall ($R@k$) and ranking quality ($N@k$) for $k \in \{1, 3, 5, 7, 10\}$, macro-averaged across task families (§5.3).

5 Results

5.1 ALFWorld Results

We first evaluate whether retrieved insights improve online agent execution in ALFWorld, where the query changes after every observation and action, and on WebShop (§5.2), where the agent must search product pages and complete purchases. Table 2 reports both metrics under the same retrieval protocol.

InsightEmb transfers to embodied control. All InsightEmb variants outperform the Base embedding model on test success rate in Table 2. With both BUNDLE and ATOMIC insights, InsightEmb achieves about a 5-point improvement over Base. For BUNDLE, per-game-level retrieval turnover, step counts, and zero-score rates are in Appendix Table 11 (§G.2). Since InsightEmb is trained exclusively on mathematical reasoning data, these gains are entirely from cross-domain transfer. Comparison with an ALFWorld-trained in-domain retriever is deferred to §6.3.

Per-task-type breakdown. Appendix Table 12 (within §H) breaks down test success rates by ALFWorld task type. InsightEmb leads on 4 of 6 types, with the largest gains on *clean* and *find_two*, indicating a general structural-matching gain rather than a single-task shortcut.

Mechanistic analysis. Appendix §G–§N analyzes why retrieval helps on ALFWorld: per-game statistics, qualitative cases, topical vs. procedural rule matching, and step-conditioned retrieval dynamics. InsightEmb shows a better retrieval diversity, step counts, and lower zero-score rates, and a state-aware transition from search to verification and state transformation.

5.2 WebShop Results

WebShop results are summarized in Table 2 (right column). We report average task score as a percentage, which reflects partial credit for attribute matching under the AgentBench evaluation protocol.

InsightEmb transfers to web shopping. On WebShop, retrieval quality matters: BUNDLE with Base (18.42%) falls well below the no-insight baseline (31.03%), while InsightEmb achieves a better performance (BUNDLE 31.74%, ATOMIC 32.05%). InsightEmb outperforms Base on both corpora (e.g., +72% relative on BUNDLE). Per-game-level statistics for BUNDLE are in Appendix Table 11 (§G.3).

Qualitative patterns. Manual inspection of divergent games reveals three recurring patterns. Appendix O walks through representative games for each pattern with side-by-side Base vs. InsightEmb trajectories:

1. **Variant selection awareness.** InsightEmb’s insights guide the agent to explicitly select product variants (color, size) before purchasing, while

Base frequently skips this step, resulting in partial scores instead of perfect scores.

2. **Loop prevention.** Base gets stuck in search–browse–back loops for 21–50 steps; InsightEmb’s insights about session management and error recovery help the agent break out of unproductive cycles.
3. **Procedural sequencing.** InsightEmb retrieves insights that encode a sequential workflow (search → verify → select variants → buy), while Base retrieves topically relevant but procedurally vague insights.

These show InsightEmb performs *procedural matching*, retrieving insights that address the agent’s current bottleneck.

5.3 Static Skill Retrieval on SRA-Bench

Following the static protocol in §4.3, we test whether math-only training improves *task-to-skill* matching before any action model runs. Tables 3 and 4 report macro-averaged and per-family results over the 636 gold skills and 26,262 distractors. InsightEmb improves over Base on every cutoff, with small gains at R@1 and substantially larger gains at higher cutoffs (+8.90 R@10). This pattern indicates that the training primarily improves the ranking of relevant skills beyond the first position, which is especially useful when an agent can inspect or condition on multiple retrieved skills.

The only clear negative outlier is MedCalcBench at small cutoffs in Table 4. Medical calculation skills are highly template-like and lexically specialized: correct retrieval often depends on exact score names, disease terms, biomarkers, and units. Base appears to preserve these entity-level cues, whereas InsightEmb’s cross-domain training favors more abstract structural matching and may downweight such terminology at rank 1—consistent with prior SRA-Bench results where MedCalcBench already reaches about 90% R@1 after BM25 reranking, suggesting lexical generation is less of a bottleneck there. In contrast, theorem proving, tool use, contest math, and code generation benefit from abstraction-aware ranking because queries and useful skills can differ substantially in wording while sharing procedural structure. This matters for skill-augmented pipelines where an LLM reranker only inspects a bounded candidate set (e.g., BM25 top-50): families with weak lexical overlap are the harder bottleneck, and gains on LogicBench and CHAMP are especially encouraging in that regime. Appendix L discusses the MedCalcBench outlier,

lexical vs. structural retrieval bottlenecks, and implications for reranker-limited skill pipelines.

6 Analysis and Comparisons

The results above establish that math-trained InsightEmb improves dynamic agent execution and static skill retrieval over the Base embedder. This section asks *why*: we compare reasoning-oriented embedders and alternative insight generators, validate in-domain fine-tuning against cross-domain training, then analyze whether the learned geometry matches our structural-analogy claim.

6.1 Scaling with Comparison to Reasoning-Oriented Retrievers

A natural question is whether reasoning-oriented retrievers trained for static, query–document matching can serve as drop-in replacements for InsightEmb in agentic settings. Using the same ALFWorld and WebShop agent protocol as Table 2, we compare Base, InsightEmb, and two reasoning-oriented embedding baselines: (i) **Llama-NV-Embed-Reasoning-3B** (NVIDIA, 2026) (3.2B; Llama-3.2-3B), the top-ranked open-source single-model embedder on BRIGHT; and (ii) **ReasonIR-8B** (Shao et al., 2025) (8B; Llama-3.1-8B), trained with contrastive learning on synthetic reasoning-intensive pairs, showing the setting closest to ours. Both differ from InsightEmb (4B; Qwen3-Embedding-4B (Zhang et al., 2025)), which is trained only on math data. Beyond the top-1 results in Table 2, we sweep retrieval budgets $k \in \{1, 3, 5\}$ only for all four embedders on both environments and both insight corpora (BUNDLE and ATOMIC). At $k=5$, top-BUNDLE retrieval already prepends about 25 instruction-level rules to the agent prompt (roughly five rules per bundle), so we do not evaluate larger k . Figure 2 plots ALFWorld (**left**) and WebShop (**right**) performances as a function of k , using the arithmetic mean of BUNDLE and ATOMIC at each $k \in \{1, 3, 5\}$. We also ablate Stage 1 by training a variant that drops partial-trajectory queries while keeping the same I^+ and I^- sets (*no half-traj*; dashed curve, InsightEmb color). Appendix A provides separate ALFWorld/WebShop top- k curves by corpus.

On the corpus-averaged curves in Figure 2, InsightEmb surpasses ReasonIR and Llama-NV-Reasoning in both ALFWorld and WebShop at each $k \in \{1, 3, 5\}$. This gap is not explained by stronger static matching alone: reasoning-oriented retriev-

Embedding	R@1	R@3	R@5	R@7	R@10	N@1	N@3	N@5	N@7	N@10
Base	30.33	40.84	46.40	49.83	54.13	33.82	37.62	39.96	41.23	42.64
InsightEmb	31.37	46.39	54.25	58.56	63.03	36.29	42.00	45.29	46.92	48.42
Δ	+1.04	+5.55	+7.85	+8.73	+8.90	+2.47	+4.38	+5.33	+5.69	+5.78

Table 3: SRA-Bench macro-average retrieval results across six task families. Queries use task information and candidates use the full skill content. $R@k$ is recall at k and $N@k$ is normalized discounted cumulative gain at k . Table 4 breaks down per-task-family results at @1 and @10; the macro-average is computed by averaging those task families equally.

Task family	n	Base				InsightEmb			
		R@1	N@1	R@10	N@10	R@1	N@1	R@10	N@10
TheoremQA	747	48.19	48.19	74.43	60.63	58.10	58.10	86.61	72.37
LogicBench	760	1.58	1.58	40.39	19.53	6.32	6.32	39.34	20.18
ToolQA	1,430	16.22	16.22	44.83	28.84	29.65	29.65	62.45	45.69
CHAMP	223	13.86	19.73	38.34	26.97	19.06	29.15	52.58	38.48
MedCalcBench	1,100	92.73	92.73	98.18	95.80	63.27	63.27	98.18	81.52
BigCodeBench	1,140	9.42	24.47	28.62	24.09	11.85	31.23	39.04	32.27

Table 4: Per-task SRA-Bench retrieval results at @1 and @10. Queries use task information and candidates use full skill content. Bold marks the better value within each task family; ties are bolded for both embeddings.

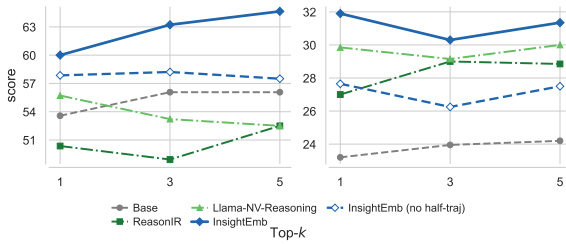


Figure 2: Top- k scaling for reasoning-oriented retrievers. **Left:** ALFWorld success rate. **Right:** WebShop average task score. Curves show the arithmetic mean of BUNDLE and ATOMIC retrieval at each $k \in \{1, 3, 5\}$ for Base, ReasonIR, Llama-NV-Reasoning, and InsightEmb (solid). The dashed line (InsightEmb color) is the *no half-traj* ablation, which removes partial-trajectory queries from Stage 1.

ers are trained for *static abductive retrieval* (a fixed query; retrieve a passage whose relevance is judged by explanatory or logical support), whereas agentic insight retrieval is *dynamic action-intent retrieval*, an evolving state that retrieve the rule that unblocks the *current* procedural step. InsightEmb is trained for the latter geometry; ReasonIR and Llama-NV-Reasoning are not.

Effect of partial-trajectory queries. The *no half-traj* ablation removes pairs whose query q is a partial (incomplete) solution trace, leaving query-only and full-trajectory anchors. The dashed curve in Figure 2 lies between Base and full InsightEmb on both environments. At the k that

maximizes corpus-averaged InsightEmb on each environment, the step from *no half-traj* to full InsightEmb accounts for about 83% of the total Base-to-InsightEmb gain on ALFWorld and about 49% on WebShop, indicating that much of InsightEmb’s gains over Base comes from partial-trajectory situation anchors with shared I^+ and I^- . Together with Figure 2, this helps further explain why InsightEmb outperforms static reasoning retrievers that are not trained on such mid-inference anchors.

6.2 Robustness to Alternative Insight Generators

All main results in §5 use insight corpora distilled from agent trajectories with DeepSeek-R1 (DeepSeek-AI, 2025). To test whether our gains depend on that particular generator, we repeat the dynamic-agent evaluation with corpora produced by GPT-5.2 (OpenAI, 2025) instead, while keeping the action model (Qwen3-8B) and the same top- k retrieval protocol as above. Appendix D.1 compares GPT-5.2 and DeepSeek-R1 corpus sizes and average rule lengths; Appendix D plots the full top- k robustness curves for both environments and corpora.

Figure 3 plots the arithmetic mean of BUNDLE and ATOMIC at each $k \in \{1, 3, 5\}$. Results show that for both ALFWorld and WebShop, InsightEmb consistently outperforms Base, supporting that improvement persists under alternative insight generators.

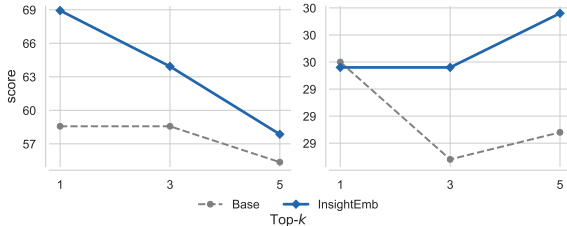


Figure 3: Robustness check with GPT-5.2-generated insight corpora. **Left:** ALFWorld. **Right:** WebShop. Curves average BUNDLE and ATOMIC at each k for Base and InsightEmb. Separate per-corpus curves appear in Appendix D.

Corpus	Embedding	ALFWorld
BUNDLE	Base	54.29
	ALF (In-domain)	57.86
	InsightEmb	60.71
ATOMIC	Base	55.00
	ALF (In-domain)	57.86
	InsightEmb	59.29

Table 5: ALFWorld test success rate for an ALFWorld-trained in-domain retriever (BUNDLE only) vs. Base and math-trained InsightEmb (top-1 retrieval; Qwen3-8B agent). The in-domain model is trained and evaluated on the same environment.

6.3 Validating the Structural-Analogy Claim

Math-to-agentic transfer. Table 1 predicts *bidirectional* transfer: math training should improve agentic retrieval (tested in §5), and with this hypothesis, embodied training should improve math retrieval also. We validate both directions and compare against the natural alternative of in-environment fine-tuning.

Finetuning With ALFWorld A standard alternative to cross-domain training is to fine-tune the embedder on ALFWorld situation-to-insight pairs. We train this *In-domain* retriever on the same backbone (Qwen3-Embedding-4B (Zhang et al., 2025)), using the same agent stack, 140 test games, and top-1 dynamic retrieval as §4. Table 5 contrasts it with Base and InsightEmb. InsightEmb exceeds the in-domain retriever on both BUNDLE and ATOMIC despite using no ALFWorld training data, suggesting that structural diversity in math supervision matters more than task-matched ALFWorld fine-tuning. Appendix F argues that math supervision offers broader structural diversity and abstraction range than single-environment ALFWorld fine-tuning.

Task	Base	InsightEmb	ALF
Number Theory*	77.22	79.81	78.33
Geometry*	54.07	59.29	57.41
Counting & Probability*	74.47	77.43	75.53
Algebra	86.69	88.21	87.81

Table 6: Math *insight* retrieval accuracy. ALF = ALFWorld-trained model. *In-domain (same domains as training, held-out test split with separate insight pool).

Agentic-to-math transfer. For the reverse direction, we evaluate all model variants on *math insight retrieval*, retrieve the correct heuristic rule for a held-out math problem. Table 6 presents accuracy on four math domains. Three domains (number theory, geometry, and counting & probability) overlap with the training distribution (same domains, different problems from the test split with a separate insight corpus), while algebra is entirely out-of-distribution and never appears in training. InsightEmb consistently improves over Base on the held-out math test set and the unseen math domain. This confirms that the contrastive curriculum does not sacrifice in-domain math retrieval while learning the claimed geometry. More importantly, the ALFWorld-trained model (ALF) also improves over Base on math insight retrieval across all four domains. This reverse-direction gain supports Table 1: the geometry is shared, not a one-way math-to-agent shortcut.

7 Conclusion

We studied *agentic insight retrieval*: at each decision step, an agent must retrieve an abstract rule that resolves its *current* procedural bottleneck, not merely text that similar with the observation. We introduced **InsightEmb**, a contrastive framework that learns goal-conditioned action-intent matching from mathematical reasoning data alone. On ALFWorld and WebShop, InsightEmb raises *end-task* success when a LLM agent runs with dynamic top- k insight injection. On SRA-Bench, it raises *retrieval recall* and nDCG for task-to-skill matching. InsightEmb outperforms the Base embedder and an ALFWorld-trained in-domain retriever on embodied control, beating strong reasoning-oriented embedders. Mechanistic analyses further show state-aware retrieval shifts. Together, these results support a simple design claim: math heuristic retrieval and agentic insight retrieval share the same progress-oriented matching structure in learning an action-intent retrieval geometry.

Limitations. Our dynamic agent experiments use Qwen3-8B as the action model and DeepSeek-R1 (with a GPT-5.2 robustness check) for insight distillation; validating on additional environments and LLMs would further strengthen the generality claim. Although SRA-Bench isolates static retrieval quality, dynamic agentic retrieval still lacks gold labels for which insight is optimal at each state; downstream success therefore remains an important but indirect measure of retrieval utility and closely aligns with our objective. For Stage 1 partial-trajectory queries, we reuse the same I^+ and I^- sets as for query-only and full-trajectory pairs on each trajectory subset (§6.1). Generating trajectory-specific (I^+ , I^-) pairs for partial states, so that positives and negatives reflect what is helpful *at that point* in the rollout, would likely improve performance and state awareness, but we did not pursue this variant due to time constraints and instead kept the same insight sets across query forms.

References

- Jianlyu Chen, Junwei Lan, Chaofan Li, Defu Lian, and Zheng Liu. 2025. [ReasonEmbed: Enhanced text embeddings for reasoning-intensive document retrieval](#). *arXiv preprint arXiv:2510.08252*. Describes the BGE-Reasoner-Embed embedding models (released under the BGE-Reasoner framework).
- Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 2020. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning*.
- DeepSeek-AI. 2025. [DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning](#). *arXiv preprint arXiv:2501.12948*.
- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. [Mind2Web: Towards a generalist agent for the web](#). In *Advances in Neural Information Processing Systems*.
- Yao Fu, Dong-Ki Kim, Jaekyeom Kim, Sungryull Sohn, Lajanugen Logeswaran, Kyunghoon Bae, and Honglak Lee. 2024. [AutoGuide: Automated generation and selection of context-aware guidelines for large language model agents](#). In *Advances in Neural Information Processing Systems*.
- Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. REALM: Retrieval-augmented language model pre-training. In *Proceedings of the 37th International Conference on Machine Learning*.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, and 1 others. 2024. [AgentBench: Evaluating LLMs as agents](#). In *International Conference on Learning Representations*.
- Bodhisattwa Prasad Majumder, Bhavana Dalvi Mishra, Peter Jansen, Oyvind Tafjord, Niket Tandon, Li Zhang, Chris Callison-Burch, and Peter Clark. 2023. CLIN: A continually learning language agent for rapid task adaptation and generalization. *arXiv preprint arXiv:2310.10134*.
- Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. 2023. [MTEB: Massive text embedding benchmark](#). In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*.
- NVIDIA. 2026. [Llama-NV-Embed-Reasoning-3B](#). Hugging Face model card. 3.2B-parameter reasoning embedding model built on Llama-3.2-3B; top-ranked open-source retriever on BRIGHT.
- OpenAI. 2025. [Update to GPT-5 system card: GPT-5.2](#). Technical report.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Vishvak Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, and 1 others. 2024. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *Proceedings of the Twelfth International Conference on Learning Representations*.
- Qwen Team. 2025. [Qwen3 technical report](#). *arXiv preprint arXiv:2505.09388*.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023.

- Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*.
- Rulin Shao, Rui Qiao, Varsha Kishore, Niklas Muennighoff, Xi Victoria Lin, Daniela Rus, Bryan Kian Hsiang Low, Sewon Min, Wen-tau Yih, Pang Wei Koh, and Luke Zettlemoyer. 2025. [ReasonIR: Training retrievers for reasoning tasks](#). *arXiv preprint arXiv:2504.20595*.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*.
- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2021. ALFWorld: Aligning text and embodied environments for interactive learning. In *Proceedings of the Ninth International Conference on Learning Representations*.
- Hongjin Su, Weijia Shi, Jungo Kasai, Yizhong Wang, Yushi Hu, Mari Ostendorf, Wen-tau Yih, Noah A. Smith, Luke Zettlemoyer, and Tao Yu. 2023. One embedder, any task: Instruction-finetuned text embeddings. In *Findings of the Association for Computational Linguistics: ACL 2023*.
- Hongjin Su, Howard Yen, Mengzhou Xia, Weijia Shi, Niklas Muennighoff, Varsha Kishore, Luke Zettlemoyer, Wen-tau Yih, Danqi Chen, and Tao Yu. 2025. [BRIGHT: A realistic and challenging benchmark for reasoning-intensive retrieval](#). In *International Conference on Learning Representations*.
- Weihang Su, Jianming Long, Qingyao Ai, Yichen Tang, Changyue Wang, Yiteng Tu, and Yiqun Liu. 2026. [Skill retrieval augmentation for agentic AI](#). *arXiv preprint arXiv:2604.24594*.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*.
- Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*.
- Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-Pack: Packaged resources to advance general Chinese embedding. *arXiv preprint arXiv:2309.07597*.
- Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul N. Bennett, Junaid Ahmed, and Arnold Overwijk. 2021. Approximate nearest neighbor negative contrastive learning for dense text retrieval. In *Proceedings of the Ninth International Conference on Learning Representations*.
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. WebShop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems*.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the Eleventh International Conference on Learning Representations*.
- Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. 2022. STaR: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems*.
- Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. [Qwen3 embedding: Advancing text embedding and reranking through foundation models](#). *arXiv preprint arXiv:2506.05176*.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. [ExpEL: LLM agents are experiential learners](#). In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xianyi Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Soheil Alikhani, and 1 others. 2024. [WebArena: A realistic web environment for building autonomous agents](#). In *International Conference on Learning Representations*.

A Reasoning-Oriented Retriever Top- k Details

Figure 2 in the main text averages BUNDLE and ATOMIC at each $k \in \{1, 3, 5\}$. Figures 4 and 5 show the per-corpus breakdown. The dashed *no half-traj* curve (InsightEmb color) removes partial-trajectory queries from Stage 1 while keeping the same I^+ and I^- sets.

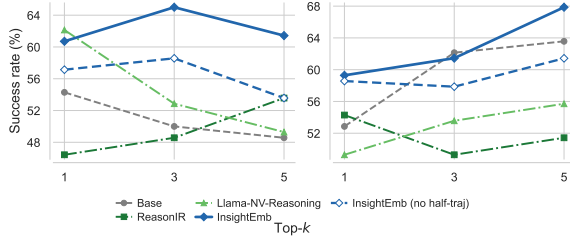


Figure 4: ALFWorld test success rate vs. retrieval budget $k \in \{1, 3, 5\}$ for Base, ReasonIR, Llama-NV-Reasoning, InsightEmb, and the *no half-traj* ablation (dashed, InsightEmb color).

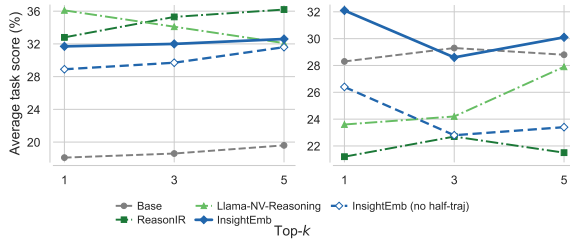


Figure 5: WebShop average task score at each k for the same models and corpora.

B Insight Corpus Construction

This appendix documents how agentic and math insight corpora are built and how they are converted into InsightEmb training data.

B.1 Pipeline Overview

Step 1: Collect training trajectories. For each math training problem (or ALFWorld/WebShop training game), we run the task model five times on the training split and log full interaction histories, following trajectory-collection practice in self-improvement methods (Zelikman et al., 2022; Zhao et al., 2024). Successful and failed rollouts are grouped by problem, retaining all five responses per instance together with labels indicating which rollouts succeeded.

Step 2: Distill insights. We apply a multi-subset genetic search over trajectory subsets. For each subset, an LLM (DeepSeek-R1 (DeepSeek-AI, 2025)

for main experiments; GPT-5.2 (OpenAI, 2025) for the robustness check) receives a distillation prompt (§B.2) and produces a BUNDLE insight: a numbered list of general, task-agnostic rules. Each candidate insight is validated on the problems represented in that subset: we prepend the rules to each problem’s query, re-run the solver, and keep the insight only if it improves solve rate over a no-insight baseline on those problems. Chromosomes that yield helpful bundles are kept; ATOMIC rules are obtained by splitting each bundle into individual numbered rules (chain-of-thought removed). For ALFWorld and WebShop, each trajectory step is first compressed into a one-sentence summary before distillation.

Step 3: Build Stage 1 embedding training pairs. Validated bundle and atomic insights are paired with math problems in two query forms: query-only (problem statement) and query+trajectory (problem with full or partial chain-of-thought). Each pair assigns I^+ to insights that improved solve rate on that problem in the subset-validation check above, and I^- to insights that did not (Equation (1)), then filters pairs to ensure enough contrastive negatives per training group.

Step 4: Build Stage 2 CoT training pairs. We construct situation-to-experience pairs from structurally similar solved problems, pairing each query with helpful and unhelpful demonstration trajectories.

B.2 Insight-Generation Prompts

Math (successful trials). Used when all trajectories in a subset succeeded:

You are an advanced reasoning agent that can create rules based on forming new critiques of past task trajectories.

You will be given successful tasks trials in which you are doing math derivation. Here are the trails. {trajs} By examining the successful trials, you can create the new rules are GENERAL and HIGH LEVEL insights of the successful trials or proposed way of Thought so they can be used as helpful tips to different tasks in the future. Have an emphasis on tips that help the agent perform better Thought and Action. Do not mention the trials in the rules because all the rules should be GENERALLY APPLICABLE. Each rule should be concise and easy to follow. Any operation can be used MULTIPLE times.

Math (mixed success and failure). Used when a subset contains both correct and incorrect solutions; trajectories are wrapped as <correct_trial_i> or <wrong_trial_i>:

You are an advanced reasoning agent that can create rules based on forming new critiques of past task trajectories.

You will be given multiple tasks trials in which you are doing math derivation. Here are the trails. {trajs} By examining and contrasting between failed and successful trials, you can create the new rules are GENERAL and HIGH LEVEL critiques of the failed trials or proposed way of Thought so they can be used as helpful tips or to avoid similar failures when encountered with different questions in the future. Have an emphasis on critiquing how to perform better Thought and Action. Do not mention the trials in the rules because all the rules should be GENERALLY APPLICABLE. Each rule should be concise and easy to follow. Any operation can be used MULTIPLE times.

ALFWorld (successful trials). After step-level summarization, used when all trajectories in a subset succeeded:

You are an advanced reasoning agent that can create rules based on forming new critiques of past AlfWorld task trajectories.

You will be given successful AlfWorld task trials involving multi-turn conversations with an environment. Here are the trajectories: {trajs}

By examining the successful trials, you can create new rules that are GENERAL and HIGH LEVEL insights about successful AlfWorld task completion strategies. Focus on insights that help the agent perform better in multi-turn dialogue with environments.

Key aspects to consider: - Effective communication strategies with the environment - Problem-solving approaches in interactive settings - Handling of multi-step reasoning in dialogue - Adaptation to environment feedback - Efficient exploration and action selection

Do not mention specific trials in the rules because all rules should be GENERALLY APPLICABLE to AlfWorld tasks. Each rule should be concise and easy to follow. Any operation can be used MULTIPLE times.

ALFWorld (mixed success and failure).

You are an advanced reasoning agent that can create rules based on forming new critiques of past AlfWorld task trajectories.

You will be given multiple AlfWorld task trials involving multi-turn conversations. Here are the trajectories: {trajs}

By examining and contrasting between failed and successful trials, you can create new rules that are GENERAL and HIGH LEVEL critiques of failed strategies or proposed ways of thinking. Focus on critiquing how to perform better in multi-turn dialogue with environments.

Key aspects to consider: - Communication breakdowns with the environment - Ineffective problem-solving approaches - Poor handling of multi-step reasoning - Failure to adapt to environment feedback - Inefficient exploration strategies

Do not mention specific trials in the rules because all rules should be GENERALLY APPLICABLE. Each rule should be concise and easy to follow.

WebShop (successful trials). After step-level summarization, used when all trajectories in a subset succeeded:

You are an advanced reasoning agent that can create rules based on forming new critiques of past WebShop task trajectories.

You will be given successful WebShop task trials involving multi-turn conversations with a web shopping environment. Here are the trajectories: {trajs}

By examining the successful trials, you can create new rules that are HIGH LEVEL insights about successful WebShop task completion strategies. Focus on insights that help the agent perform better in multi-turn dialogue with web shopping environments.

Key aspects to consider: - Effective search query formulation strategies - Product selection and comparison approaches - Handling of product attributes (size, color, price, etc.) - Navigation strategies (search, click, back, buy) - Efficient exploration of product listings - Matching user requirements to product descriptions - Decision-making on when to buy vs. continue searching

Do not mention specific trials in the rules. Each rule should be concise and easy to follow. Any operation can be used MULTIPLE times.

WebShop (mixed success and failure).

You are an advanced reasoning agent that can create rules based on forming new critiques of past WebShop task trajectories.

You will be given multiple WebShop task trials involving multi-turn conversations. Here are the trajectories: {trajs}

By examining and contrasting between failed and successful trials, you can create new rules that are HIGH LEVEL critiques of failed strategies or proposed ways of thinking. Focus on critiquing how to perform better in multi-turn dialogue with web shopping environments.

Key aspects to consider: - Ineffective search query formulation - Poor product selection and comparison decisions - Failure to match product attributes to requirements - Inefficient navigation patterns - Premature or delayed purchase decisions - Misinterpretation of product descriptions - Failure to adapt search strategy based on results

Do not mention specific trials in the rules. Each rule should be concise and easy to follow.

Insight validation (math). For a subset of trajectories, the distillation prompt produces a candidate rule set; we then test it on every problem in that subset by prepending the rules to the problem query and re-running the solver:

Below is an instruction that describes a task. Write a response that appropriately completes the request and wrap the final answer inside \boxed{{}}.

{generated insight rules} #### Problem: {problem from the subset}

Solution: Let's think step by step.

For each problem q in the subset, we record whether the solver succeeds with vs. without the prepended rules; the insight is assigned to I^+ on problems where it helps and to I^- on problems where it does not, which defines the Stage 1 contrastive labels.

B.3 Example Generated Insights

Math (counting & probability).

- **Define Variables & Relationships:** Start by defining variables for unknown quantities and establish mathematical relationships based on given conditions to structure the problem.
- **Complementary Probability for “Not” Events:** When calculating the probability of an event **not** occurring (e.g., not sitting together), compute the probability of the complementary event and subtract it from 1.
- **Leverage Standard Combinatorial Formulas:** Recognize scenarios like distributing identical items (stars and bars) or non-colinear points (triangles in a cube) and apply the appropriate formulas directly.

ALFWorld.

1. **Maintain a Goal-Driven Action Sequence:** Successful agents follow a clear path toward the goal, while failed ones get sidetracked.
2. **Adapt Exploration Based on Feedback:** If an object isn’t found where expected, systematically check other plausible locations instead of repeating the same action.
3. **Verify Object Presence Before Interaction:** Avoid assuming an object is present; check first to prevent unnecessary actions.
4. **Prioritize Common Storage Areas First:** Check typical locations (like countertops or stove burners) mentioned in the task before deep cabinet search.

WebShop.

1. Formulate search queries with all key user specifications (e.g., “gluten free chai orca spice flavor under \$20”).
2. If initial results are irrelevant, refine search terms by adding or rephrasing keywords.
3. Before selecting a product, cross-check all attributes (price, features, specs) against user requirements.
4. Use systematic navigation: proceed through pages methodically and backtrack if no matches are found.
5. Compare multiple products on the same page to identify the best match before clicking.

C Insight Corpus Statistics

Main experiments (§4, §5) use DeepSeek-R1 (DeepSeek-AI, 2025) generated insight corpora for ALFWorld and WebShop at two granularities: BUNDLE (multi-rule summaries with chain-of-thought) and ATOMIC (single rules split from bundles). Table 7 reports corpus size and length statistics for each environment; these corpora are referenced throughout the dynamic-agent evaluation.

D Alternative Insight-Generator Robustness

D.1 Corpus Statistics

Table 8 summarizes GPT-5.2 (OpenAI, 2025)-generated insight corpora used in the robustness check (§6.2). BUNDLE files contain one multi-rule summary per past trajectories subset; ATOMIC files expand each bundle into individual rules. Compared with the DeepSeek-R1 (DeepSeek-AI, 2025) corpora in Table 7, GPT-5.2 bundles are shorter on average while atomic rules are similar in length.

D.2 Top- k Scaling Details

Figure 3 in the main text averages BUNDLE and ATOMIC at each k . Figures 6 and 7 show the per-corpus breakdown (Qwen3-8B agent; same protocol as Table 2).

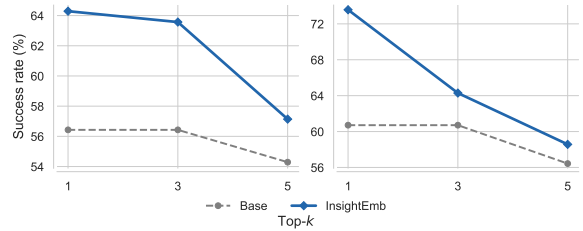


Figure 6: ALFWorld success rate vs. $k \in \{1, 3, 5\}$ with GPT-5.2-generated insights.

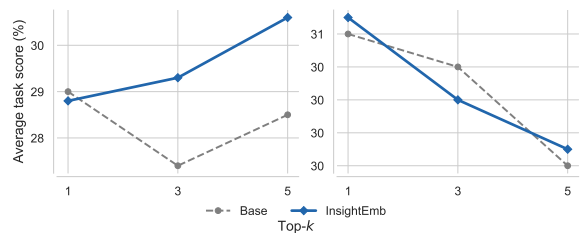


Figure 7: WebShop average task score vs. $k \in \{1, 3, 5\}$ with GPT-5.2-generated insights.

Environment	Corpus	# Insights	Avg Words	Range
ALFWorld	BUNDLE	2,419	494	90–1,114
	ATOMIC	21,916	25	1–365
WebShop	BUNDLE	501	549	176–1,064
	ATOMIC	5,212	24	1–371

Table 7: DeepSeek-R1 insight corpus statistics (main experiments). BUNDLE = multi-rule insights; ATOMIC = individual rules extracted from bundles. Range = min–max words per insight.

Environment	Corpus	# Insights	Avg Words	Range
ALFWorld	BUNDLE	700	314	178–500
	ATOMIC	8,410	25	11–58
WebShop	BUNDLE	700	358	198–532
	ATOMIC	10,162	23	4–54

Table 8: GPT-5.2 insight corpus statistics. Range = min–max words per non-empty insight; WebShop has 10 empty bundle summaries (word count 0). Atomic = single rules split from bundles.

E Training Data Statistics

InsightEmb is trained only on mathematical reasoning data (Stage 1: situation-to-insight; Stage 2: situation-to-experience; §3.3–3.4). Table 9 summarizes contrastive training examples per stage. Stage 1 combines **bundle** insight pairs (7,740 unique rules in the insight pool) and **atomic** pairs (22,243 unique rules). “Query” = query-only samples; “Traj” = query concatenated with a full or partial CoT trajectory. Table 10 gives the per-domain Stage 1 breakdown (bundle + atomic).

Stage	Granularity	Total	Query	Traj
Stage 1	Bundle	5,463	1,917	3,546
	Atomic	6,487	2,308	4,179
	Combined	11,950	4,225	7,725
Stage 2 (CoT)	—	2,896	2,329	567

Table 9: Training data statistics. Stage 1 totals combine bundle and atomic insight pairs.

Math Domain	Bundle	Atomic	Stage 1 Total
Counting & Probability	1,736	2,647	4,383
Number Theory	2,220	2,400	4,620
Geometry	1,507	1,440	2,947
Total	5,463	6,487	11,950
Math Domain	Stage 2 (CoT)		
Counting & Probability	966		
Number Theory	966		
Geometry	964		
Total	2,896		

Table 10: Per-domain training counts. Stage 1 = bundle + atomic insight pairs; Stage 2 = CoT experience pairs.

F Why Math Training Outperforms In-Domain Fine-Tuning

As shown in §6.3 in Table 5, the ALFWorld-trained retriever underperforms math-trained InsightEmb despite task-matched supervision. Two factors explain this gap. First, *data diversity*: math training covers three structurally distinct reasoning domains, each with its own abstraction patterns, while ALFWorld training data is limited to a single environment with repetitive task structures. Second, *abstraction range*: mathematical insights span from concrete formulas to high-level meta-strategies, providing a richer gradient of abstraction levels for the contrastive loss to exploit. These factors suggest that the quality of contrastive training for insight retrieval depends more on the *structural diversity* of the training domain than on its topical proximity to the target domain.

G Per-Game-Level Agent Statistics

This appendix supplements the aggregate results in Table 2 with *per-game-level* analysis: for each ALFWorld or WebShop test game, we log the final task score, the number of environment steps until termination, and how many *distinct* insight blocks were injected over the course of that game. All numbers use Qwen3-8B with top-1 *dynamic* insight retrieval on BUNDLE summary insights—the same protocol as the BUNDLE rows in Table 2. The goal is to show that InsightEmb’s gains are not only a higher mean success rate, but also more varied retrieval within each game, shorter successful trajectories, and fewer complete failures.

G.1 Metrics and computation

Each evaluated game is one independent rollout from the initial observation until success, failure, or the 50-step cap. At every step, the current agent state is embedded and the top-1 insight is retrieved from the pre-encoded BUNDLE corpus; that text block is prepended to the action prompt.

Steps counts the environment steps in that rollout.

Distinct insights per game counts how many *different* insight strings were injected over the inference. Retrieval is recomputed at *every* step (up to 50), so each step could return a different top-1 neighbor, the count is therefore bounded by the number of steps (≤ 50). The statistic measures how often the top-1 neighbor *changes*, not corpus size.

Avg. task score is the mean of per-game-level outcomes: on ALFWorld, 1 for a win and 0 otherwise (equivalent to success rate); on WebShop, the environment’s `task_score` $\in [0, 1]$ converted to a percentage.

Zero-score games are failed ALFWorld rollouts or WebShop games with `task_score` = 0.

Games with >1 unique is the count of games where retrieval changed at least once.

G.2 ALFWorld (140 test games)

On ALFWorld, InsightEmb improves every column in Table 11.

Retrieval turnover. Retrieval runs at every step, so distinct insights per game is bounded by steps (≤ 50). Base nevertheless uses only 1.82 distinct strings on average (max 4) while InsightEmb averages 5.79 (max 22). The comparison is about how often top-1 changes as the state evolves, not about injecting a new corpus block every step.

Efficiency. Mean steps decrease from 30.3 to 28.2 and the *median* drops from 30 to 24, so at least half of InsightEmb games finish in ≤ 24 steps whereas Base’s median sits at the cap for unsuccessful search-heavy runs. Shorter medians co-occur with higher success: InsightEmb avoids prolonged cabinet-by-cabinet loops documented in Appendix H.3.

Failures. Zero-score games (failed rollouts) decrease from 64 to 55, showing gains even in completely failed games.

G.3 WebShop (500 test games)

Retrieval turnover. With per-step top-1 retrieval over up to 50 steps, distinct insights per game could be as large as the step count; Base averages 1.85 (max 4) and InsightEmb 5.98 (max 18). InsightEmb changes the retrieved block in 487/500 games (97%) vs. 295 (59%) for Base, so different steps can surface search, variant-selection, and checkout rules from the BUNDLE corpus.

Efficiency. Average steps drop from 40.8 to 33.9 (mean rollout ~ 41 steps) while both medians remain at 50, because many games still hit the cap; InsightEmb shortens trajectories it completes rather than shifting the median below the horizon. Appendix O illustrates loop-heavy Base runs vs. earlier termination when purchase rules are retrieved.

Failures. Zero-score games decrease from 309 to 258 (−51), the gain comes from converting full failures into partial or full purchase credit.

H ALFWorld Per-Task and Mechanistic Analysis

This section expands the ALFWorld mechanistic claims summarized in §5: per-task-type breakdown, multi-granularity invariance, qualitative retrieval examples, rule-level topical vs. procedural matching, and step-conditioned retrieval dynamics.

H.1 Per-Task-Type Breakdown

Table 12 supplements the aggregate ALFWorld results in Table 2 by reporting test success counts for each ALFWorld task family (ATOMIC corpus, top-1 retrieval, Qwen3-8B). Counts are out of the games listed in parentheses per type; bold marks the higher count between Base and InsightEmb.

H.2 Multi-Granularity Query Invariance

Our Stage 1 design enforces that query-only, partial-trajectory, and full-trajectory versions of the same problem all retrieve the same insight. This directly maps to the agentic setting: at step 0 the agent has only the task description (analogous to query-only), at mid-game it has partial action history (partial trajectory), and at late steps it has extensive history (full trajectory). The multi-granularity training ensures that the correct insight is retrievable at every stage of task execution, not just at the beginning.

Metric	ALFWorld		WebShop	
	Base	InsightEmb	Base	InsightEmb
# games	140	140	500	500
# zero-score	64	55	309	258
Avg. task score	54.3%	60.7%	18.42%	31.74%
Avg. steps	30.3	28.2	40.8	33.9
Median steps	30	24	50	50
Distinct insights / game ($\leq$ steps)	1.82	5.79	1.85	5.98
Games with >1 distinct insight	85	134	295	487
Max distinct insights / game	4	22	4	18

Table 11: Per-game-level statistics (Qwen3-8B, top-1 dynamic retrieval, BUNDLE insights). Failed ALFWorld games typically reach the 50-step cap.

Task Type	Base	Ours
clean (22)	27	41
cool (26)	62	69
heat (19)	42	53
put (48)	71	71
examine (17)	71	53
find_two (8)	12	38
Overall	55.0	59.3

Table 12: ALFWorld success rates by task type (top-1 retrieval, Qwen3-8B). Base = ATOMIC/Base; Ours = ATOMIC/InsightEmb. Numbers in parentheses indicate total games per type.

H.3 Qualitative Example: Insight-Guided Search

Figure 8 illustrates how InsightEmb’s retrieved insights lead to better action selection on a representative task (“put a hot potato in fridge”).

Task: “put a hot potato in fridge” Base model retrieves (ATOMIC/Base): <i>“Apply Task-Specific Physics Reasoning — Heat transfers require closed appliances. Objects in containers remain inaccessible until explicit retrieval.”</i> → Agent focuses on heating procedure but cannot locate the potato → Timeout at 50 steps InsightEmb retrieves (ATOMIC/InsightEmb): <i>“Prioritize Task-Relevant Locations First — Always check object containers/destinations mentioned in the task (e.g., stoveburner, sidetable).”</i> → Agent locates potato → heat → place in fridge → Success in 37 steps
--

Figure 8: Qualitative comparison on Game 0 (test, top-5 retrieval). The base model’s insight describes the heating *procedure* but not *where to find* the potato; InsightEmb retrieves a search-priority rule that addresses the actual bottleneck.

The base model’s insight is topically relevant but strategically vacuous—it does not indicate *where* to

find the potato. InsightEmb retrieves a structurally matched search-priority rule, directly determining the agent’s first action. This pattern—generic vs. structurally specific retrieval—recurs across the 16 games that InsightEmb wins uniquely under top-1 retrieval (see also §I).

H.4 Topical vs. Procedural Retrieval: Rule-Level Analysis

Because each ATOMIC insight is a single rule, we can examine which *type* of rule each embedding model prioritizes for the same query. We classify retrieved rules into functional categories—WHERE (search strategy), HOW (state-change procedures), PLACE (destination logic), and VERIFY (state checks)—with GPT-5.2 (OpenAI, 2025).

The Base model retrieves rules about *how to heat*—“Apply Task-Specific Physics Reasoning,” “Apply Thermodynamic Context Filtering,” “Appliance Function Mapping”—which describe the heating *procedure* but not *where to find the potato*. InsightEmb retrieves rules about *where to search*—“Prioritize Task-Relevant Locations First,” “Prioritize Direct Affordances,” “Prioritize Immediate Target Interaction”—which address the actual bottleneck: the agent’s first action must be to *locate* the target object.

This reveals a key difference in how the two models understand task structure:

- **Base** matches the query to rules that share the task’s *topic* (heating → heating rules). This is *topical matching*.
- **InsightEmb** matches the query to rules that address the task’s *current bottleneck* (the agent hasn’t found the object yet → search rules). This is *action-intent matching*: retrieving the rule implied by what the agent needs to do next.

The action-intent match is correct: in ALFWorld, the agent must first *find* the target object before it

can apply any state transformation. InsightEmb has learned this sequential dependency from the structural parallel in math, where a problem must first be *understood* (matched to the right strategy) before it can be *solved* (executed step by step).

I ALFWorld Qualitative Case Studies

We present two additional examples where InsightEmb succeeds while Base fails (top-1 retrieval).

Game 9: “clean some ladle and put it in countertop.” InsightEmb checks cabinet 1–3, then sinkbasin 1, opens cabinet 4, finds the ladle on countertop 1, cleans it at the sink, and places it on the countertop—completing in **12 steps**. Base loops through cabinets and the sink without taking the ladle, timing out at 50 steps. This illustrates task-focused search: InsightEmb reaches the sink and visible surfaces early, while Base’s generic “check fridges first” heuristic delays locating the ladle.

Game 33: “put a cool lettuce in countertop.” InsightEmb opens the fridge, scans countertop 1–3, takes the lettuce from countertop 3, cools it in the fridge, and places it on the countertop—completing in **10 steps**. Base opens the fridge but then wanders through drawers and cabinets, never retrieving the lettuce, timing out at 50 steps. Base’s retrieved summary rule prioritizes “common storage areas” broadly; InsightEmb’s rule set emphasizes checking task-mentioned surfaces (countertops) before deep cabinet search.

J Training Setup

We fine-tune Qwen3-Embedding-4B (Zhang et al., 2025), building on contrastive embedding practice (Wang et al., 2022; Su et al., 2023), using DeepSpeed ZeRO-3 on 8 GPUs. Both stages use learning rate 1×10^{-5} with cosine scheduling, per-device batch size 8 with 2 gradient accumulation steps, temperature $\tau=0.01$, training group size 11, maximum passage length 1,536 tokens, 6 epochs, and warmup ratio 0.1. Stage 2 initializes from the Stage 1 checkpoint. Table 13 lists the full configuration.

K SRA-Bench Query and Skill Encoding

We evaluate SRA-Bench with the same Qwen3-style query prefix used for Base and InsightEmb at inference. For each instance, the query text is

the full question field from the benchmark (background context plus task description). Skills are encoded without an instruction prefix.

Retrieval instruction. The fixed task description prepended to every query is:

Given an agent task question, retrieve the reusable skill that helps solve the task. The skill may describe a theorem, logic pattern, tool workflow, medical calculator, math concept, or software library.

Query format. Each query is embedded as:

Instruct: Given an agent task question, retrieve the reusable skill that helps solve the task. The skill may describe a theorem, logic pattern, tool workflow, medical calculator, math concept, or software library.
Query:{question}

where {question} is the full SRA-Bench instance text. This matches the Qwen3 Instruct: . . . Query: convention used for our embedding models.

Skill (passage) format. Each candidate skill in the mixed corpus (636 gold and 26,262 distractor skills) is embedded as:

```
{name}
{description}

{content}
```

using the name, description, and content fields from the benchmark corpus JSON. We rank all corpus skills against each query by cosine similarity over normalized embeddings.

L SRA-Bench Retrieval Analysis

MedCalcBench is the only task family where InsightEmb underperforms Base at small cutoffs (§5.3, Table 4). Medical calculation skills are highly template-like, lexically specialized, and domain-specific: correct retrieval often depends on exact medical score names, disease terms, biomarker or variable names, units, and other clinical terminology. The Base embedding model appears to preserve these fine-grained biomedical and entity-level cues, whereas InsightEmb’s cross-domain training encourages more abstract structural matching and may therefore downweight or smooth over such terminology at rank 1. This interpretation is also consistent with the original SRA-Bench results, where MedCalcBench already achieves about 90% R@1 and over 90% R@10 after reranking the BM25 top-50 candidates with

different reranker models, indicating that lexical candidate generation is less of a bottleneck for this task. In contrast, task families such as theorem proving, tool use, contest math, and code generation benefit from abstraction-aware ranking because the query and useful skill can differ substantially in wording while sharing a reasoning or procedural structure. This distinction matters in practical retrieval-augmented skill selection pipelines: an LLM reranker can only inspect a limited candidate set, such as the BM25 top-50 skills, due to context-window constraints. Thus, task families for which lexical retrieval fails to place the correct skill within this window constitute the more important bottleneck. From this perspective, the strong results on LogicBench and CHAMP are especially encouraging, because they require more semantic or reasoning-oriented matching between task descriptions and skill content, where surface lexical overlap is weak. InsightEmb therefore improves the recall of relevant skills before reranking, expanding the effective coverage of downstream LLM-based selection.

M Hyperparameters

Both training stages share the settings in Appendix J. Table 13 lists the full configuration.

Hyperparameter	
Base model	Qwen3-Emb-4B
Learning rate	1×10^{-5}
LR scheduler	Cosine
Warmup ratio	0.1
Batch size (per device)	8
Grad. accumulation	2
Training epochs	6
Temperature τ	0.01
Training group size	11
Max passage length	1,536
Precision	BF16
GPUs	8
DeepSpeed	ZeRO-3
Seed	3407

Table 13: Training hyperparameters for both stages.

N Step-Conditioned Retrieval Details

The rule-level analysis above examines *which* rules are retrieved; we now examine *when* they are retrieved. We extract the top-1 retrieved ATOMIC insight at every step of all 140 test games, classify each rule with GPT-5.2 (OpenAI, 2025), and plot the per-step category mix in Figure 9 with 3-step rolling average. For readability we merge SEARCH

& LOCATE with NAVIGATION into **Search**, and STATE TRACKING with TASK PLANNING into **State checking** (six categories total; definitions and examples below).

Base vs. InsightEmb dynamics. Base shows almost no step-wise change: **Search** is 49.7% on average over steps 1–4 vs. 51.0% over steps 20–25 ($\Delta=+1.3$ pp), **State Transform** 23.0% $\rightarrow$ 23.6%, and **Verification** 14.8% $\rightarrow$ 16.1%. InsightEmb instead shows a clear *state-aware* shift along the episode: **Search** falls from 78.1% (steps 1–4) to 61.7% (steps 20–25; $\Delta=-16.4$ pp), while **Verification** rises from 16.9% to 31.5% ($\Delta=+14.6$ pp) and **State Transform** from 1.9% to 5.9% ($\Delta=+3.9$ pp). That is, InsightEmb retrieval tracks procedural phase—heavy search while the object is still missing, then more checks and state-change rules as the agent approaches completion—whereas Base remains roughly uniform.

Category definitions and illustrative rules.

Each label denotes the *procedural role* of a single atomic rule (not the ALFWorld task type):

Search

Rules for finding the target or reaching the right place before manipulation. *Example*: “Prioritize Task-Relevant Locations First — Always check object containers/destinations mentioned in the task (e.g., stoveburner, sidetable).”

State Transform

Rules for changing object state (clean, heat, cool) at the correct appliance. *Example*: “Apply Contextual Cleaning Protocol — When ‘clean’ is specified in the task, automatically prioritize sinkbasin interaction after obtaining the target object, before any placement attempts.”

Verification

Rules for confirming prerequisites or completion before the next action. *Example*: “Ensure Cooling Completion Before Placement — Verify the cooling action (e.g., fridge interaction) is fully executed and acknowledged by the environment before attempting to move the object to its final destination.”

State checking

Rules for tracking progress, ordering sub-goals, or maintaining awareness of what was already searched or carried. *Example*: “Maintain a Dynamic Search Tracker — Systematically record searched locations and their contents to avoid

redundant checks and focus exploration on un-examined areas.”

Error Recovery

Rules for escaping loops or reformulating strategy after failed attempts. *Example:* “Implement Loop Detection — Establish a threshold (3–5 attempts) for repeated unsuccessful navigation patterns, then trigger systematic re-evaluation of object location hypotheses.”

Placement

Rules for putting the object at its final receptacle. *Example:* “Container Proximity Hierarchy — When storing objects, first navigate to the nearest valid container type specified in the task (drawer/cabinet) before considering distant alternatives.”

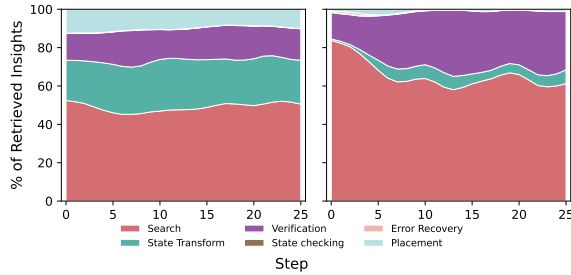


Figure 9: Per-step retrieved insight category distribution (ATOMIC, test, top-1 dynamic retrieval; 3-step rolling average). **Left:** Base embedder. **Right:** InsightEmb. Base changes little across steps; InsightEmb shifts from Search toward Verification and State Transform.

O WebShop Qualitative Examples

We present illustrative examples from the WebShop evaluation where InsightEmb with ATOMIC insights succeeds while Base fails, or achieves substantially higher task scores. These examples demonstrate the three qualitative patterns identified in §5.2: variant selection awareness, loop prevention, and procedural sequencing.

O.1 Example 1: Office Chair (Game 226, Atomic)

Task: “Find me height adjustable, high density, easy install, easy assemble home office chairs for living room with color: type 7-pink, and price lower than 120.00 dollars.”

InsightEmb: WON in 6 steps (score = 1.0).
Base: LOST in 3 steps (score = 0.857).

InsightEmb insight:

Progressive Filter Strategy — When initial search yields partial matches, systematically add missing requirements to subsequent queries.

Base insight:

Precision-First Search — Always include all critical attributes in the initial search query using AND logic for better filtering.

Action comparison. Both models find the same product (b08p81rfz4), but:

- **InsightEmb:** search → click product → check features → go back → **select “type 7-pink”** → buy now ✓
- **Base:** search (with AND syntax) → click product → **buy now immediately** (without selecting color variant) ✗

Analysis. InsightEmb’s “Progressive Filter Strategy” guides the agent to inspect product features and select the correct variant before purchasing. Base’s “Precision-First Search” focuses on query formulation but does not instruct variant selection, causing premature purchase with a partial score.

O.2 Example 2: Women’s Sweater (Game 479, Atomic)

Task: “Find me women’s sweaters with relaxed fit, long sleeve with color: light heather grey, and size: large, and price lower than 50.00 dollars.”

InsightEmb: WON in 8 steps (score = 1.0).
Base: LOST in 3 steps (score = 0.600).

InsightEmb insight:

Attribute Cross-Checking — Before any click, verify that the product matches all specified attributes (price, color, size, etc.).

Action comparison. Both find the same product (b07dkgjr74), but:

- **InsightEmb:** search → explore first result → back to search → re-search → click product → **select “light heather grey”** → **select “large”** → buy now ✓
- **Base:** search → click product → **buy now immediately** (without selecting color or size) ✗

Analysis. This is the clearest example of the *variant selection awareness* pattern. InsightEmb’s “Attribute Cross-Checking” rule explicitly guides the agent to verify and select each product variant before purchasing. Base’s generic search-formulation rule does not encode this procedural constraint, and the agent buys immediately without selecting the required color and size variants.

O.3 Example 3: Dining Set (Game 287, Atomic)

Task: “Find me button tufted, mid century, high density, easy assemble dining sets with solid wood, wood frame for dining room with color: light grey, and price lower than 250.00 dollars.”

InsightEmb: WON in 4 steps (score = 1.0).

Base: LOST in 50 steps (score = 0.000).

This is the most dramatic example ($\Delta = +1.000$).

InsightEmb insight:

Precision-First Search — Always include all critical attributes (type, color, size, price limit) in the initial search query using AND logic for better filtering.

Action comparison.

- **InsightEmb:** search (comprehensive query) → click product (b09gy58gdh) → **select “light grey”** → buy now ✓ (4 steps)
- **Base:** search → back to search → re-search → back to search → re-search → click next → click next → back to search → ... (50 steps, never clicks a product to purchase)

Analysis. InsightEmb’s concise insight produces an effective search query that finds the product immediately. Base gets stuck in a search-refine-next-back loop for all 50 steps, illustrating the *loop prevention* failure: without a clear recovery strategy, the agent exhaustively re-searches without ever committing to a product.

O.4 Example 4: iPad Case (Game 190, Bundle)

Task: “Find me compatible apple online game services with case cover with color: coast coconut trees, and price lower than 50.00 dollars.”

InsightEmb: WON in 6 steps (score = 1.0).

Base: LOST in 50 steps (score = 0.000).

Action comparison.

- **InsightEmb:** search “apple game case cover” → back to search → **re-search “iPad case coconut tree design”** → click product (b07vvvdrcx) → **select “coast coconut trees”** → buy now ✓ (6 steps)
- **Base:** search “Apple case cover” → click next → next → next → next → back to search → re-search → next → next → next → ... (50 steps, navigating pages without clicking any product)

Analysis. InsightEmb’s agent reformulates the search query after the first attempt fails, switching from the literal task wording to a more product-oriented query (“iPad case coconut tree design”). Base’s agent repeatedly paginates through results without reformulating, illustrating the *loop prevention* pattern: InsightEmb’s retrieval surfaces strategies for query refinement and error recovery.

O.5 Example 5: Men’s Sleepwear (Game 166, Atomic)

Task: “Find me men’s sleep & lounge with long sleeve, elastic waistband for daily wear with color: multi 10, and size: xx-large, and price lower than 80.00 dollars.”

InsightEmb: WON in 9 steps (score = 1.0).

Base: LOST in 7 steps (score = 0.667).

InsightEmb insight:

Compound Feature Recognition — For combined requirements, search for the most distinctive pairing first, then check secondary attributes in product details.

Action comparison.

- **InsightEmb:** search → click product (b09nd9dp7j) → check description → back to search → re-search with exact attributes → click same product → **select “multi 10”** → **select “xx-large”** → buy now ✓
- **Base:** search → click product (b09nd8p2qr) → check features → back to search → re-search → click same product → **buy now without selecting variants** ✗

Analysis. InsightEmb’s “Compound Feature Recognition” guides the agent through a verify-then-select workflow: first confirm the product matches, then explicitly select each variant before purchase. Base’s agent finds a similar product but purchases without selecting the required color and size, resulting in a partial score. This combines *variant selection awareness* with *procedural sequencing*: InsightEmb encodes a sequential workflow (search → verify → select variants → buy).

O.6 Summary of WebShop Qualitative Patterns

Across all divergent games, three consistent patterns emerge. The qualitative cases above and the main WebShop column in Table 2.

1. **Variant selection awareness** (Games 226, 479, 393, 166): InsightEmb consistently guides the

agent to select product variants (color, size) before purchasing. Base frequently skips this step, resulting in partial scores (0.600–0.857) instead of perfect scores. This is the single most impactful behavioral difference.

2. **Loop prevention** (Games 287, 190): Base gets stuck in search–browse–back loops for 50 steps, scoring zero. InsightEmb’s insights about query reformulation and error recovery help the agent find the product and complete the purchase.
3. **Procedural sequencing** (Game 166): InsightEmb retrieves insights that encode a sequential workflow (search → verify → select variants → buy), while Base retrieves topically relevant but procedurally vague insights. This mirrors the ALFWorld finding where InsightEmb performs *procedural matching* rather than *topical matching*.